

VL-16: NP-vollständige Zahlprobleme

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

WS 2024, RWTH

Wdh.: NP-schwer & NP-vollständig

Definition

Ein Problem L^* ist **NP-schwer**, falls $\forall L \in NP : L \leq_p L^*$

Ein Problem L^* ist **NP-vollständig**, falls $L^* \in NP$ und L^* ist NP-schwer.

Satz

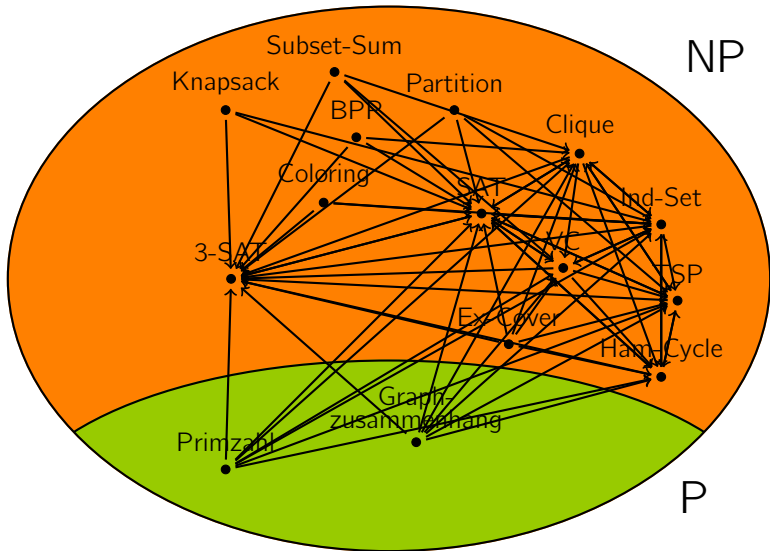
Wenn L^* NP-vollständig ist, dann gilt: $L^* \in P \Rightarrow P = NP$

Unter der Annahme $P \neq NP$ (Standardannahme) besitzt also kein NP-vollständiges Problem einen polynomiellen Algorithmus.

Kochrezept:

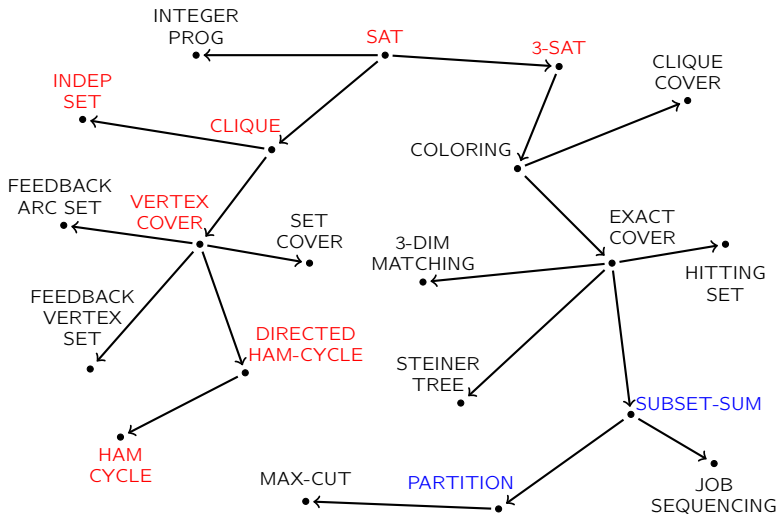
1. Man zeige $L \in NP$.
2. Man wähle eine NP-vollständige Sprache L^* .
3. **(Reduktionsabbildung)**: Man konstruiere eine Funktion f , die Instanzen von L^* auf Instanzen von L abbildet.
4. **(Polynomielle Zeit)**: Man zeige, dass f in polynomieller Zeit berechnet werden kann.
5. **(Korrektheit)**: Man beweise, dass f tatsächlich eine Reduktion ist. Für $x \in \{0, 1\}^*$ gilt $x \in L^*$ genau dann, wenn $f(x) \in L$.

Wdh.: Die Komplexitätslandschaft



Warnung: Dieser Abbildung liegt die Annahme $P \neq NP$ zu Grunde.

Wdh.: Landkarte mit Karp's 20 Reduktionen



Vorlesung VL-16

Einige NP-vollständige Zahlprobleme

- NP-Vollständigkeit von SUBSET-SUM
- NP-Vollständigkeit von PARTITION
- NP-Vollständigkeit von Bin Packing und Knapsack
- Pseudo-polynomielle Zeit
- Starke NP-Schwere
- Das THREE-PARTITION Problem

NP-Vollständigkeit von SUBSET-SUM

SUBSET-SUM: Definition

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Beispiel: Eingabe für SUBSET-SUM

Zahlen $1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192$
und Zielsumme $b = 9999$

SUBSET-SUM: Definition

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Beispiel: Eingabe für SUBSET-SUM

Zahlen $1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192$
und Zielsumme $b = 9999$

Satz

SUBSET-SUM ist NP-vollständig.

SUBSET-SUM: Reduktion

Satz

SUBSET-SUM ist NP-vollständig.

Beweis:

- SUBSET-SUM liegt in NP
- Wir zeigen $3\text{-SAT} \leq_p \text{SUBSET-SUM}$
- Die Boole'sche Formel φ in 3-CNF sei eine Instanz von 3-SAT
- Die Formel hat Klauseln $c_1, \dots, c_m$ mit den Variablen $x_1, \dots, x_n$

In der Reduktion arbeiten wir mit Dezimalzahlen mit jeweils $n + m$ Ziffern.

Die k -te Ziffer einer Zahl z bezeichnen wir dabei mit $z(k)$.

Reduktion (1): Var-Zahlen

Wir definieren:

$$S^+(i) = \{j \in \{1, \dots, m\} \mid \text{Klausel } c_j \text{ enthält Literal } x_i\}$$

$$S^-(i) = \{j \in \{1, \dots, m\} \mid \text{Klausel } c_j \text{ enthält Literal } \bar{x}_i\}$$

Reduktion (1): Var-Zahlen

Wir definieren:

$$S^+(i) = \{j \in \{1, \dots, m\} \mid \text{Klausel } c_j \text{ enthält Literal } x_i\}$$

$$S^-(i) = \{j \in \{1, \dots, m\} \mid \text{Klausel } c_j \text{ enthält Literal } \bar{x}_i\}$$

Für jede Boolesche Variable x_i mit $1 \leq i \leq n$ erzeugen wir zwei entsprechende Var-Zahlen a_i^+ und a_i^- mit den folgenden Ziffern:

$$a_i^+(i) = 1 \quad \text{und für alle } j \in S^+(i) : a_i^+(n+j) = 1$$

$$a_i^-(i) = 1 \quad \text{und für alle } j \in S^-(i) : a_i^-(n+j) = 1$$

Alle anderen Ziffern in diesen Dezimaldarstellungen sind 0.

Reduktion: Beispiel

Als Beispiel betrachten wir die Formel

$$(x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \bar{x}_3 \vee \bar{x}_4)$$

Die folgenden Var-Zahlen werden erzeugt:

$$a_1^+ = 1000\textcolor{red}{10}$$

$$a_1^- = 1000\textcolor{red}{00}$$

$$a_2^+ = 0100\textcolor{red}{11}$$

$$a_2^- = 0100\textcolor{red}{00}$$

$$a_3^+ = 0010\textcolor{red}{10}$$

$$a_3^- = 0010\textcolor{red}{01}$$

$$a_4^+ = 0001\textcolor{red}{00}$$

$$a_4^- = 0001\textcolor{red}{01}$$

Reduktion (2): Dummy-Zahlen

- Wir definieren für jede Klausel c_j zwei entsprechende **Dummy-Zahlen** d_j und d'_j .
- Dummy-Zahlen haben nur an der Ziffernposition $n + j$ eine Ziffer 1; alle anderen Ziffern sind 0.

Fortsetzung des Beispiels

Wir betrachten wieder die Formel

$$(x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \bar{x}_3 \vee \bar{x}_4)$$

Die Dummy-Zahlen für die beiden Klauseln lauten dann:

$$d_1 = 0000\mathbf{10}$$

$$d'_1 = 0000\mathbf{10}$$

$$d_2 = 0000\mathbf{01}$$

$$d'_2 = 0000\mathbf{01}$$

Reduktion (3): Zielsumme

Die **Zielsumme** b definieren wir folgendermassen:

- $b(k) = 1$ für $1 \leq k \leq n$,
- $b(k) = 3$ für $n + 1 \leq k \leq n + m$.

Abschluss des Beispiels

Wir betrachten wieder die Formel

$$(x_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \bar{x}_3 \vee \bar{x}_4)$$

Die Zielsumme lautet dann:

$$b = 111133$$

Reduktion: Illustration

Mögliche Zahlwerte für eine Formel mit n Variablen und m Klauseln:

	1	2	3	...	n	$n+1$	$n+2$	...	$n+m$
a_1^+	1	0	0	...	0	1	0	...	...
a_1^-	1	0	0	...	0	0	0	...	...
a_2^+	0	1	0	...	0	0	1	...	...
a_2^-	0	1	0	...	0	1	0	...	...
a_3^+	0	0	1	...	0	1	1	...	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
a_n^+	0	0	0	...	1	0	0	...	...
a_n^-	0	0	0	...	1	0	1	...	...
d_1	0	0	0	...	0	1	0	...	0
d'_1	0	0	0	...	0	1	0	...	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
d_m	0	0	0	...	0	0	0	...	1
d'_m	0	0	0	...	0	0	0	...	1
b	1	1	1	...	1	3	3	...	3

Beweis (1): Keine Carry-Overs

- Für jede Dezimalstelle $i \in \{1, \dots, n\}$ gilt: Nur zwei der Var-Zahlen und Dummy-Zahlen haben an dieser Stelle die Ziffer 1; alle anderen Zahlen haben an dieser Stelle die Ziffer 0.
- Für jede Dezimalstelle $i \in \{n+1, \dots, n+m\}$ gilt: Nur fünf der Var-Zahlen und Dummy-Zahlen haben an dieser Stelle die Ziffer 1; alle anderen Zahlen haben an dieser Stelle die Ziffer 0.

Beweis (1): Keine Carry-Overs

- Für jede Dezimalstelle $i \in \{1, \dots, n\}$ gilt: Nur zwei der Var-Zahlen und Dummy-Zahlen haben an dieser Stelle die Ziffer 1; alle anderen Zahlen haben an dieser Stelle die Ziffer 0.
- Für jede Dezimalstelle $i \in \{n+1, \dots, n+m\}$ gilt: Nur fünf der Var-Zahlen und Dummy-Zahlen haben an dieser Stelle die Ziffer 1; alle anderen Zahlen haben an dieser Stelle die Ziffer 0.

Beobachtung: Keine Carry-Overs

Wird eine beliebige Menge von Var-Zahlen und Dummy-Zahlen addiert, so treten bei den Sprüngen von einer Dezimalstelle zur nächsten niemals Additionsüberträge auf.

Beweis (2): Laufzeit der Reduktion

- Die SAT Instanz φ besteht aus n Variablen und m Klauseln.
Die Eingabelänge ist $\geq m + n$.
- Die konstruierte SUBSET-SUM Instanz besteht aus $2n + 2m + 1$ Dezimalzahlen mit je $m + n$ Dezimalstellen.
- Die Reduktion wird in polynomieller Zeit $O((m + n)^2)$ durchgeführt.

Beweis (3): Korrektheit, Teil A

Lemma A: Formel φ erfüllbar $\Rightarrow$ SUBSET-SUM Instanz ist lösbar

Es gibt eine erfüllende Belegung x^* für die Formel φ .

Beweis (3): Korrektheit, Teil A

Lemma A: Formel φ erfüllbar $\Rightarrow$ SUBSET-SUM Instanz ist lösbar

Es gibt eine erfüllende Belegung x^* für die Formel φ .

- Falls $x_i^* = 1$, so wählen wir a_i^+ aus; andernfalls wählen wir a_i^-
- Die Summe der ausgewählten Var-Zahlen bezeichnen wir mit A
- Da für jedes $i \in \{1, \dots, n\}$ entweder a_i^+ oder a_i^- ausgewählt wurde, gilt $A(i) = 1$

Beweis (3): Korrektheit, Teil A

Lemma A: Formel φ erfüllbar $\Rightarrow$ SUBSET-SUM Instanz ist lösbar

Es gibt eine erfüllende Belegung x^* für die Formel φ .

- Falls $x_i^* = 1$, so wählen wir a_i^+ aus; andernfalls wählen wir a_i^-
- Die Summe der ausgewählten Var-Zahlen bezeichnen wir mit A
- Da für jedes $i \in \{1, \dots, n\}$ entweder a_i^+ oder a_i^- ausgewählt wurde, gilt $A(i) = 1$
- Ausserdem gilt $A(n+j) \in \{1, 2, 3\}$ für $1 \leq j \leq m$, weil in jeder Klausel ein oder zwei oder drei Literale erfüllt sind.

Beweis (3): Korrektheit, Teil A

Lemma A: Formel φ erfüllbar $\Rightarrow$ SUBSET-SUM Instanz ist lösbar

Es gibt eine erfüllende Belegung x^* für die Formel φ .

- Falls $x_i^* = 1$, so wählen wir a_i^+ aus; andernfalls wählen wir a_i^-
- Die Summe der ausgewählten Var-Zahlen bezeichnen wir mit A
- Da für jedes $i \in \{1, \dots, n\}$ entweder a_i^+ oder a_i^- ausgewählt wurde, gilt $A(i) = 1$
- Ausserdem gilt $A(n+j) \in \{1, 2, 3\}$ für $1 \leq j \leq m$, weil in jeder Klausel ein oder zwei oder drei Literale erfüllt sind.
- Falls $A(n+j) \in \{1, 2\}$, so wählen wir zusätzlich d_j und/oder d'_j aus, um die Ziffer 3 an Ziffernposition $n+j$ der Summe zu erhalten.

Also gibt es eine Teilmenge mit der gewünschten Zielsumme b .

Beweis (4): Korrektheit, Teil B

Lemma B: SUBSET-SUM Instanz ist lösbar $\Rightarrow$ Formel φ erfüllbar

Es gibt eine Teilmenge K_A der Var-Zahlen (mit Summe A) und eine Teilmenge K_D der Dummy-Zahlen (mit Summe H), die sich zur Zielsumme b aufaddieren; also: $A + H = b$.

Beweis (4): Korrektheit, Teil B

Lemma B: SUBSET-SUM Instanz ist lösbar $\Rightarrow$ Formel φ erfüllbar

Es gibt eine Teilmenge K_A der Var-Zahlen (mit Summe A) und eine Teilmenge K_D der Dummy-Zahlen (mit Summe H), die sich zur Zielsumme b aufaddieren; also: $A + H = b$.

- Die Menge K_A enthält für jedes $i \in \{1, \dots, n\}$ genau eine der beiden Var-Zahlen a_i^+ und a_i^- ; andernfalls wäre $A(i) \neq 1$.

Beweis (4): Korrektheit, Teil B

Lemma B: SUBSET-SUM Instanz ist lösbar $\Rightarrow$ Formel φ erfüllbar

Es gibt eine Teilmenge K_A der Var-Zahlen (mit Summe A) und eine Teilmenge K_D der Dummy-Zahlen (mit Summe H), die sich zur Zielsumme b aufaddieren; also: $A + H = b$.

- Die Menge K_A enthält für jedes $i \in \{1, \dots, n\}$ genau eine der beiden Var-Zahlen a_i^+ und a_i^- ; andernfalls wäre $A(i) \neq 1$.
- Wir setzen $x_i = 1$ falls $a_i^+ \in K_A$, und andernfalls $x_i = 0$.

Beweis (4): Korrektheit, Teil B

Lemma B: SUBSET-SUM Instanz ist lösbar $\Rightarrow$ Formel φ erfüllbar

Es gibt eine Teilmenge K_A der Var-Zahlen (mit Summe A) und eine Teilmenge K_D der Dummy-Zahlen (mit Summe H), die sich zur Zielsumme b aufaddieren; also: $A + H = b$.

- Die Menge K_A enthält für jedes $i \in \{1, \dots, n\}$ genau eine der beiden Var-Zahlen a_i^+ und a_i^- ; andernfalls wäre $A(i) \neq 1$.
- Wir setzen $x_i = 1$ falls $a_i^+ \in K_A$, und andernfalls $x_i = 0$.
- Es gilt $A(n+j) \geq 1$ für $1 \leq j \leq m$.
Ansonsten wäre $A(n+j) + H(n+j) \leq A(n+j) + 2 < 3$.
- Dadurch ist sichergestellt, dass in jeder Klausel mindestens eines der Literale den Wert 1 hat.

Die Formel φ ist also erfüllbar.

NP-Vollständigkeit von PARTITION

PARTITION: Definition

Problem: PARTITION

Eingabe: Positive ganze Zahlen $a'_1, \dots, a'_n$; mit $\sum_{i=1}^n a'_i = 2A'$

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a'_i = A'$?

PARTITION ist der Spezialfall von SUBSET-SUM mit $b := (\sum a_i)/2$

PARTITION: Definition

Problem: PARTITION

Eingabe: Positive ganze Zahlen $a'_1, \dots, a'_n$; mit $\sum_{i=1}^n a'_i = 2A'$

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a'_i = A'$?

PARTITION ist der Spezialfall von SUBSET-SUM mit $b := (\sum a_i)/2$

Satz

PARTITION ist NP-vollständig.

Beweis:

- PARTITION liegt in NP
- Wir zeigen $\text{SUBSET-SUM} \leq_p \text{PARTITION}$

PARTITION: Reduktion

- Es sei $a_1, \dots, a_n \in \mathbb{N}$ und $b \in \mathbb{N}$ eine beliebige Instanz von SUBSET-SUM
- Es sei $S := \sum_{i=1}^n a_i$, und o.B.d.A. gilt $S \geq b$

Wir bilden diese SUBSET-SUM Instanz auf eine PARTITION Instanz ab, die aus den folgenden $n + 2$ Zahlen $a'_1, \dots, a'_{n+2}$ besteht:

PARTITION: Reduktion

- Es sei $a_1, \dots, a_n \in \mathbb{N}$ und $b \in \mathbb{N}$ eine beliebige Instanz von SUBSET-SUM
- Es sei $S := \sum_{i=1}^n a_i$, und o.B.d.A. gilt $S \geq b$

Wir bilden diese SUBSET-SUM Instanz auf eine PARTITION Instanz ab, die aus den folgenden $n + 2$ Zahlen $a'_1, \dots, a'_{n+2}$ besteht:

- $a'_i = a_i$ für $1 \leq i \leq n$
- $a'_{n+1} = 2S - b$ und $a'_{n+2} = S + b$

PARTITION: Reduktion

- Es sei $a_1, \dots, a_n \in \mathbb{N}$ und $b \in \mathbb{N}$ eine beliebige Instanz von SUBSET-SUM
- Es sei $S := \sum_{i=1}^n a_i$, und o.B.d.A. gilt $S \geq b$

Wir bilden diese SUBSET-SUM Instanz auf eine PARTITION Instanz ab, die aus den folgenden $n + 2$ Zahlen $a'_1, \dots, a'_{n+2}$ besteht:

- $a'_i = a_i$ für $1 \leq i \leq n$
- $a'_{n+1} = 2S - b$ und $a'_{n+2} = S + b$

Die Summe dieser $n + 2$ Zahlen beträgt $\sum_{i=1}^{n+2} a'_i = 4S$.
Daher gilt $A' := 2S$ für die PARTITION Instanz.

Die Reduktion wird in polynomieller Zeit durchgeführt.

Beweis (1): Korrektheit, Teil A

Lemma A: SUBSET-SUM Instanz lösbar $\Rightarrow$ PARTITION Instanz lösbar

- Wenn es in der SUBSET-SUM Instanz eine Teilmenge der Zahlen $a_1, \dots, a_n$ mit der Summe b gibt, so haben die entsprechenden Zahlen $a'_1, \dots, a'_n$ in der PARTITION Instanz ebenfalls die Summe b .
- Wir fügen die Zahl $a'_{n+1} = 2S - b$ zu dieser Teilmenge dazu und erhalten eine Teilmenge mit der gewünschten Zielsumme $A' = 2S$.

Beweis (2): Korrektheit, Teil B

Lemma B: PARTITION Instanz lösbar $\Rightarrow$ SUBSET-SUM Instanz lösbar

- In der Lösung der PARTITION Instanz sind die beiden Zahlen $a'_{n+1} = 2S - b$ und $a'_{n+2} = S + b$ nicht in derselben Teilmenge, da $a'_{n+1} + a'_{n+2} = 3S > 2S = A'$ gilt.
- Eine der Teilmengen besteht aus $a'_{n+1} = 2S - b$ und einer Teilmenge der Zahlen $a'_1, \dots, a'_n$ mit Gesamtsumme $A' = 2S$.
- Die entsprechenden Zahlen in der SUBSET-SUM Instanz haben dann die Summe b .

NP-Vollständigkeit von Bin Packing und Knapsack

Bin Packing

Beim Bin Packing sollen n Objekte mit Gewichten $w_1, \dots, w_n$ auf eine möglichst kleine Anzahl von Kisten mit Gewichtslimit B verteilt werden.

Problem: Bin Packing (BPP)

Eingabe: Zahlen B und $w_1, \dots, w_n \in \{1, \dots, B\}$; eine Schranke γ

Frage: Können Objekte mit den gegebenen Größen $w_1, \dots, w_n$ in γ Kisten der Grösse B gepackt werden?

Satz

Bin Packing ist NP-vollständig.

Beweis:

- Wir zeigen $\text{PARTITION} \leq_p \text{Bin Packing}$
- Wir setzen $\gamma = 2$, und $w_i = a'_i$ für $1 \leq i \leq n$, und $B = A'$ $\square$

Im Knapsack Problem sollen Objekte ausgewählt werden, die in einen Rucksack mit Gewichtsschranke $\overline{w}$ passen und den Gesamtprofit maximieren.

Problem: Knapsack (KP)

Eingabe: Natürliche Zahlen $w_1, \dots, w_n, p_1, \dots, p_n, \overline{w}, \gamma$

Frage: Existiert eine Teilmenge der Objekte mit Gesamtgewicht höchstens $\overline{w}$ und Gesamtprofit mindestens γ ?

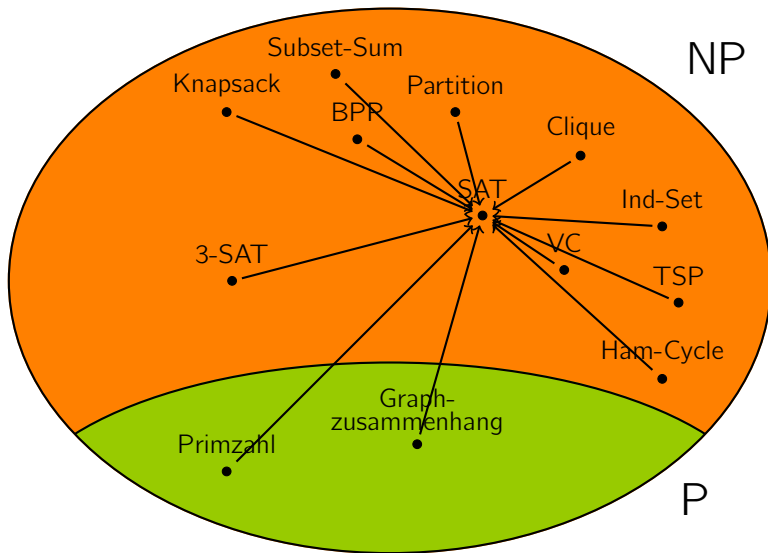
Satz

Knapsack ist NP-vollständig.

Beweis:

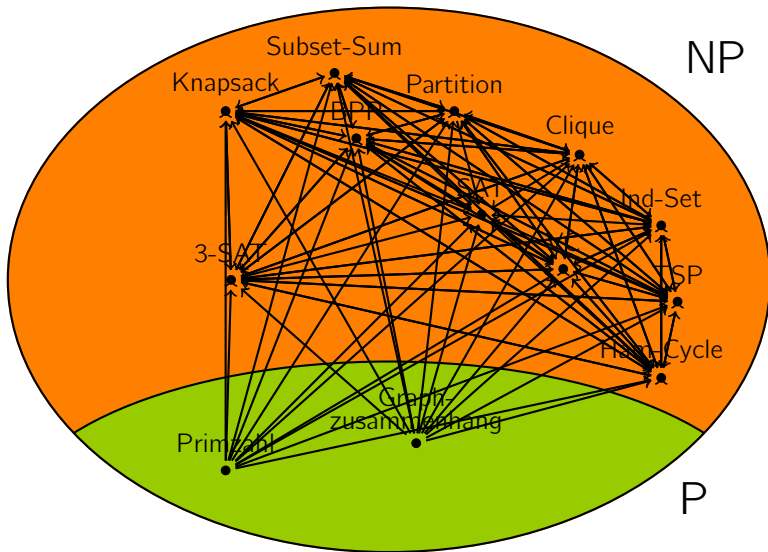
- Wir zeigen $\text{SUBSET-SUM} \leq_p \text{Knapsack}$
- Wir setzen $w_i = a_i$ und $p_i = a_i$ für $1 \leq i \leq n$, und $\overline{w} = \gamma = b$ $\square$

Die Komplexitätslandschaft



Warnung: Dieser Abbildung liegt die Annahme $P \neq NP$ zu Grunde.

Die Komplexitätslandschaft



Warnung: Dieser Abbildung liegt die Annahme $P \neq NP$ zu Grunde.

Kodierungslänge

- Es sei X ein algorithmisches Problem
- Die Laufzeit eines Algorithmus für Problem X messen wir in Abhängigkeit von der **Kodierungslänge** der Instanzen I

- Es sei X ein algorithmisches Problem
- Die Laufzeit eines Algorithmus für Problem X messen wir in Abhängigkeit von der **Kodierungslänge** der Instanzen I
- Die Kodierungslänge $|I|$ ist die Anzahl der Symbole in einer “vernünftigen” Beschreibung der Instanz I
- Kleine (polynomiell grosse) Änderungen in der Länge von derartigen Beschreibungen sind für unsere Definitionen / Sätze / Beweise / Reduktionen / Resultate irrelevant

Beispiel: Ungerichtete Graphen

Vernünftige Beschreibungen von ungerichteten Graphen $G = (V, E)$ sind

- Adjazenzlisten mit Länge $\ell_1(G) = O(|E| \log |V|)$
- Adjazenzmatrizen mit Länge $\ell_2(G) = O(|V|^2)$

Für zusammenhängende Graphen gilt:

- $\ell_1(G)$ ist polynomiell beschränkt in $\ell_2(G)$
- $\ell_2(G)$ ist polynomiell beschränkt in $\ell_1(G)$

Beispiel: Mengen und Teilmengen

Es sei X eine Grundmenge mit n Elementen. Vernünftige Beschreibungen einer Teilmenge $Y \subseteq X$ sind:

- Bitvektoren der Länge $\ell_1(Y) = n$
- Auflisten der Elemente mit Länge $\ell_2(Y) = O(|Y| \log n)$

Für dichte Mengen Y mit $\Omega(n)$ vielen Elementen gilt:

- $\ell_1(Y)$ ist polynomiell beschränkt in $\ell_2(Y)$
- $\ell_2(Y)$ ist polynomiell beschränkt in $\ell_1(Y)$

Kodierungslänge: Zahlen

Beispiel: Natürliche Zahlen

Vernünftige Beschreibungen von natürlichen Zahlen n sind

- Dezimaldarstellung mit Länge $\approx \log_{10} n$
- Binärdarstellung mit Länge $\approx \log_2 n$
- Oktaldarstellung mit Länge $\approx \log_8 n$
- Hexadezimaldarstellung mit Länge $\approx \log_{16} n$

Für alle reellen Zahlen $a, b \geq 2$ gilt: $\log_a n = \log_a b \cdot \log_b n$

Die verschiedenen Kodierungslängen unterscheiden sich daher nur um einen konstanten Faktor.

Anmerkung

Die Zahl n stellt den Wert n mit Kodierungslänge $O(\log n)$ dar. Der Wert hängt also **exponentiell** von der Kodierungslänge ab.

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Beispiel

- Für eine TSP Instanz I ist **Number**(I)

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Beispiel

- Für eine TSP Instanz I ist **Number**(I) der Wert der grössten Städtedistanz $\max_{i,j} d(i,j)$ oder der Wert γ .

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Beispiel

- Für eine TSP Instanz I ist **Number**(I) der Wert der grössten Städtedistanz $\max_{i,j} d(i,j)$ oder der Wert γ .
- Für eine SUBSET-SUM Instanz I ist **Number**(I)

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Beispiel

- Für eine TSP Instanz I ist **Number**(I) der Wert der grössten Städtedistanz $\max_{i,j} d(i,j)$ oder der Wert γ .
- Für eine SUBSET-SUM Instanz I ist **Number**(I) das Maximum der Zahlen $a_1, \dots, a_n$ und b .

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Beispiel

- Für eine TSP Instanz I ist **Number**(I) der Wert der grössten Städtedistanz $\max_{i,j} d(i,j)$ oder der Wert γ .
- Für eine SUBSET-SUM Instanz I ist **Number**(I) das Maximum der Zahlen $a_1, \dots, a_n$ und b .
- Für eine SAT Instanz I ist **Number**(I)

Zahlenwert versus Kodierungslänge (1)

Definition: Number

Für eine Instanz I eines Entscheidungsproblems bezeichnen wir mit **Number**(I) den Wert der grössten in I vorkommenden Zahl.

Beispiel

- Für eine TSP Instanz I ist **Number**(I) der Wert der grössten Städtedistanz $\max_{i,j} d(i,j)$ oder der Wert γ .
- Für eine SUBSET-SUM Instanz I ist **Number**(I) das Maximum der Zahlen $a_1, \dots, a_n$ und b .
- Für eine SAT Instanz I ist **Number**(I) das Maximum der Zahlen n und m . (Ergo: $\text{Number}(I) \leq |I|$.)

Zahlenwert versus Kodierungslänge (2)

- Für viele Entscheidungsprobleme wie zum Beispiel SAT, VC, CLIQUE, Ham-Cycle, COLORING gilt $\text{Number}(I) \leq |I|$.
- Für derartige Probleme ist $\text{Number}(I)$ **polynomiell** in der Eingabelänge beschränkt.
- Der Parameter $\text{Number}(I)$ ist nur für Probleme relevant, in denen Distanzen, Kosten, Gewichte, Längen, Profite, Zeitintervalle, Abstände, etc eine Rolle spielen.
- Zum Beispiel: TSP, SUBSET-SUM, PARTITION
- Für derartige Probleme ist $\text{Number}(I)$ nur **exponentiell** in der Eingabelänge beschränkt.

Pseudo-polynomielle Zeit und Starke NP-Schwere

Definition: Pseudo-polynomielle Zeit

Ein Algorithmus A löst ein Problem X in **pseudo-polynomieller** Zeit, falls die Laufzeit von A auf Instanzen I von X polynomiell in $|I|$ und $\text{Number}(I)$ beschränkt ist.

Definition: Pseudo-polynomielle Zeit

Ein Algorithmus A löst ein Problem X in **pseudo-polynomieller** Zeit, falls die Laufzeit von A auf Instanzen I von X polynomiell in $|I|$ und $\text{Number}(I)$ beschränkt ist.

Satz

Die Probleme SUBSET-SUM, PARTITION und Knapsack sind in pseudo-polynomieller Zeit lösbar.

Beispiel: SUBSET-SUM

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Satz

SUBSET-SUM ist in pseudo-polynomieller Zeit $O(n \cdot b)$ lösbar.

Beispiel: SUBSET-SUM

Problem: SUBSET-SUM

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$; eine ganze Zahl b

Frage: Existiert eine Indexmenge $I \subseteq \{1, \dots, n\}$ mit $\sum_{i \in I} a_i = b$?

Satz

SUBSET-SUM ist in pseudo-polynomieller Zeit $O(n \cdot b)$ lösbar.

Beweis:

- Dynamische Programmierung: Für $k = 0, \dots, n$ und $c = 0, \dots, b$ setzen wir $F[k, c] = \text{TRUE}$ genau dann, wenn es eine Indexmenge $I \subseteq \{1, \dots, k\}$ mit $\sum_{i \in I} a_i = c$ gibt.
- $F[0, c] = (c == 0)$ für $c = 0, \dots, b$
 $F[k, c] = F[k-1, c - a_k] \vee F[k-1, c]$
- Schlussendlich findet man die Antwort in $F[n, b]$

Beispiel: MAX-FLUSS

Problem: MAX-FLUSS

Eingabe: Ein gerichteter Graph $G = (V, A)$; eine Quelle $s \in V$;
eine Senke $t \in V$; Kantenkapazitäten $c : A \rightarrow \mathbb{N}$; eine Schranke γ

Frage: Existiert ein s - t -Fluss f mit Wert $f^* \geq \gamma$?

Satz

MAX-FLUSS ist in pseudo-polynomieller Zeit $O(f^*|V| + f^*|E|)$ lösbar.

Beweis: siehe DSAL Vorlesung (Ford-Fulkerson Algorithmus mit Residualnetzwerken und augmentierenden Pfaden)

Definition: Stark NP-schwer (engl.: NP-hard in the strong sense)

Ein Entscheidungsproblem X ist **stark NP-schwer**,
wenn es ein Polynom $q : \mathbb{N} \rightarrow \mathbb{N}$ gibt, sodass die Restriktion X_q
von X auf Instanzen I mit $\text{Number}(I) \leq q(|I|)$ NP-schwer ist.

Also: Das Problem X ist sogar dann NP-schwer, wenn alle Zahlen**werte**
in der Instanz I nur polynomiell gross (gemessen in $|I|$) sind.

Definition: Stark NP-schwer (engl.: NP-hard in the strong sense)

Ein Entscheidungsproblem X ist **stark NP-schwer**,
wenn es ein Polynom $q : \mathbb{N} \rightarrow \mathbb{N}$ gibt, sodass die Restriktion X_q
von X auf Instanzen I mit $\text{Number}(I) \leq q(|I|)$ NP-schwer ist.

Also: Das Problem X ist sogar dann NP-schwer, wenn alle Zahlenwerte
in der Instanz I nur polynomiell gross (gemessen in $|I|$) sind.

Übung (unter der Annahme $P \neq NP$)

Welche der folgenden Probleme sind stark NP-schwer?

- SAT und 3-SAT
- CLIQUE, INDEP-SET, Vertex Cover
- Ham-Cycle und TSP
- SUBSET-SUM und PARTITION
- Bin Packing

Stark NP-schwer vs pseudo-polynomiell

Satz

Es sei X ein stark NP-schweres Entscheidungsproblem.

Falls X pseudo-polynomiell lösbar ist, so gilt $P=NP$.

Also: Pseudo-polynomiell und stark NP-schwer schliessen einander aus
(unter unserer Standardannahme $P \neq NP$)

Stark NP-schwer vs pseudo-polynomiell

Satz

Es sei X ein stark NP-schweres Entscheidungsproblem.
Falls X pseudo-polynomiell lösbar ist, so gilt $P=NP$.

Also: Pseudo-polynomiell und stark NP-schwer schliessen einander aus
(unter unserer Standardannahme $P \neq NP$)

Beweis:

- X ist stark NP-schwer
- Ergo gibt es ein Polynom $q : \mathbb{N} \rightarrow \mathbb{N}$, für das die Restriktion X_q von X auf Instanzen I mit $\text{Number}(I) \leq q(|I|)$ NP-schwer ist.
- Ein pseudo-polynomieller Algorithmus A für X hat Laufzeit polynomiell beschränkt in $|I|$ und $\text{Number}(I)$
- Wendet man Algorithmus A auf X_q an, so ist die Laufzeit polynomiell beschränkt in $|I|$ und $q(|I|)$, und daher polynomiell beschränkt in $|I|$
- (X_q NP-schwer) und (X_q polynomiell lösbar) $\Rightarrow P=NP$

Beispiel: THREE-PARTITION

Problem: THREE-PARTITION

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$, $b_1, \dots, b_n$, und $c_1, \dots, c_n$ mit $\sum_{i=1}^n (a_i + b_i + c_i) = nS$

Frage: Gibt es zwei Permutationen α, β von $1, \dots, n$, sodass $a_{\alpha(i)} + b_{\beta(i)} + c_i = S$ für $1 \leq i \leq n$ gilt?

Beispiel: THREE-PARTITION

Problem: THREE-PARTITION

Eingabe: Positive ganze Zahlen $a_1, \dots, a_n$, $b_1, \dots, b_n$, und $c_1, \dots, c_n$ mit $\sum_{i=1}^n (a_i + b_i + c_i) = nS$

Frage: Gibt es zwei Permutationen α, β von $1, \dots, n$, sodass $a_{\alpha(i)} + b_{\beta(i)} + c_i = S$ für $1 \leq i \leq n$ gilt?

Satz (ohne Beweis)

THREE-PARTITION ist stark NP-schwer.

Übung

Zeigen Sie: Bin Packing ist stark NP-schwer.